

pg_basebackup — Physical Base Backup

Table of Contents

1. [Learning Objectives](#)
 2. [What is pg_basebackup](#)
 3. [Architecture Diagram](#)
 4. [Prerequisites and Setup](#)
 5. [Basic pg_basebackup Usage](#)
 6. [WAL Streaming During Backup](#)
 7. [Backup from Standby](#)
 8. [Backup Formats and Options](#)
 9. [Monitoring Backup Progress](#)
 10. [Using Backup for Replication Setup](#)
 11. [Production Backup Script](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions](#)
 16. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use pg_basebackup for full physical backups
 - Understand WAL streaming during backup and why it matters
 - Configure backups to run from a standby server
 - Set up new streaming replication standbys using pg_basebackup
 - Monitor backup progress and troubleshoot failures
-

What is pg_basebackup

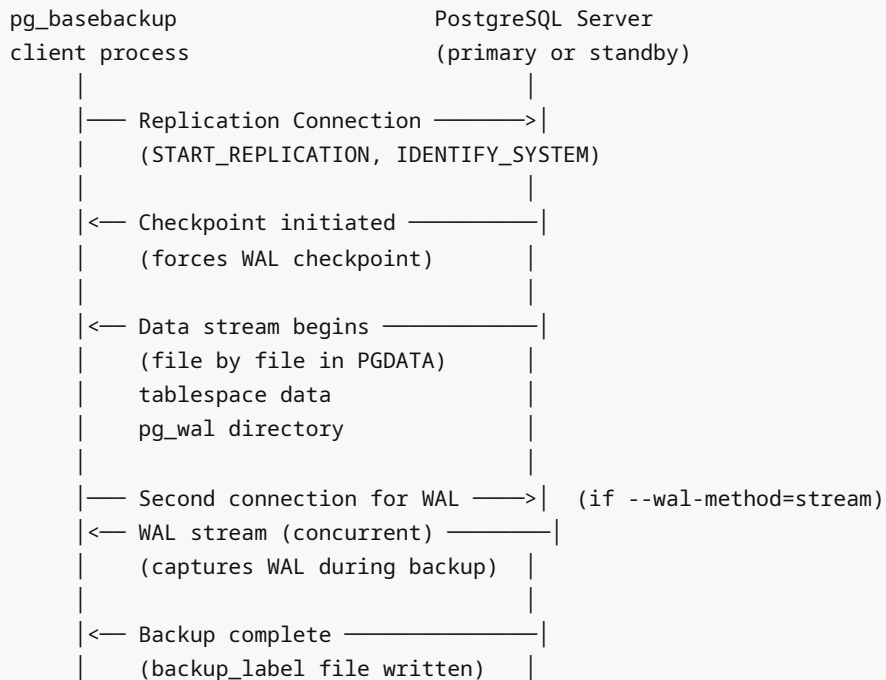
`pg_basebackup` is PostgreSQL's built-in tool for taking physical (binary) base backups. It uses the replication protocol to copy the entire data directory while the server is running.

Key characteristics:

- ✓ Online backup (server keeps running)
 - ✓ Consistent (based on WAL checkpoint)
 - ✓ Full cluster (all databases + global objects)
 - ✓ Foundation for PITR (combine with WAL archiving)
 - ✓ Used internally by Patroni for replica cloning
 - ✗ Full copy each time (no incremental)
 - ✗ Cannot exclude individual tables
 - ✗ For incremental backups: use pgBackRest
-

Architecture Diagram

pg_basebackup workflow:



Prerequisites and Setup

```
-- Required PostgreSQL configuration:
-- wal_level = replica (or logical)
-- max_wal_senders >= 2 (one for backup, one for WAL streaming)
-- max_replication_slots >= 1 (if using slots)

SHOW wal_level;          -- Must be 'replica' or 'logical'
SHOW max_wal_senders;    -- Must be >= 2

-- Create backup user
CREATE ROLE backupuser WITH REPLICATION LOGIN PASSWORD 'BackupPass123!';
-- For pg_basebackup: REPLICATION privilege is sufficient
```

```
# pg_hba.conf: allow backup user replication connection
host    replication    backupuser    192.168.1.0/24          scram-sha-256
# Or from backup server IP:
host    replication    backupuser    10.0.10.5/32           scram-sha-256
```

Basic pg_basebackup Usage

```
# Simplest form: backup to a directory
pg_basebackup \
  --host=192.168.1.100 \
```

```

--port=5432 \
--username=backupuser \
--pgdata=/backup/base/$(date +%Y%m%d) \
--progress \
--verbose

# With all recommended options
pg_basebackup \
--host=db.example.com \
--port=5432 \
--username=backupuser \
--pgdata=/backup/base/$(date +%Y%m%d_%H%M%S) \
--wal-method=stream \      # Stream WAL during backup (prevents gaps)
--checkpoint=fast \        # Don't wait for next scheduled checkpoint
--label="nightly-backup-$(date +%Y%m%d)" \ # Backup label
--progress \               # Show progress
--verbose \                # Verbose output
--no-password              # Use .pgpass or PGPASSWORD env

# Using connection string
pg_basebackup \
"host=db.example.com port=5432 user=backupuser sslmode=verify-full
sslrootcert=/etc/ssl/pg/ca.crt" \
--pgdata=/backup/base/$(date +%Y%m%d) \
--wal-method=stream \
--checkpoint=fast \
--progress

```

WAL Streaming During Backup

This is the most important `pg_basebackup` option. Without it, backups can be unusable.

The WAL Gap Problem

Without `--wal-method=stream`:

```

Backup starts: LSN 0/50000
               ...copying files... (10 minutes)
               ...WAL 0000000100000000000003 through 00000001000000000007 generated...
Backup ends:   LSN 0/80000

```

If `--wal-method=fetch` (fetches WAL at end):

The backup includes all WAL generated during the backup.
 But the primary must retain those WAL files until backup finishes.
 If `wal_keep_size` is too small → WAL files deleted → backup is BROKEN.

With `--wal-method=stream` (default and recommended):

Opens a `SECOND` connection to stream WAL continuously during the backup.
 WAL from backup start to backup end is captured in real-time.

No dependency on `wal_keep_size`.

Primary only needs `max_wal_senders >= 2` (one for stream, one for backup).

WAL Method Options

```
# --wal-method=none
# No WAL included. You must have WAL archived separately.
# Only useful if you have continuous WAL archiving already set up.
pg_basebackup --wal-method=none --pgdata=/backup/base/

# --wal-method=fetch (older default)
# Fetches WAL at end of backup. Requires primary to retain all WAL
# from start of backup. Risk: WAL recycled if wal_keep_size too small.
pg_basebackup --wal-method=fetch --pgdata=/backup/base/

# --wal-method=stream (RECOMMENDED)
# Streams WAL concurrently. Requires max_wal_senders >= 2.
# Safest option regardless of wal_keep_size.
pg_basebackup --wal-method=stream --pgdata=/backup/base/
```

Backup from Standby

Running `pg_basebackup` from a standby server reduces load on the primary.

```
# Run pg_basebackup against a hot standby
pg_basebackup \
  --host=standby.example.com \
  --port=5432 \
  --username=backupuser \
  --pgdata=/backup/base/$(date +%Y%m%d) \
  --wal-method=stream \
  --checkpoint=fast \
  --progress

# Note: The standby must have:
# hot_standby = on (already required for read queries)
# max_wal_senders >= 2 (to stream WAL to backup client)
# pg_hba.conf allowing replication from backup server

# Verify standby accepts replication connections:
psql -h standby.example.com -U backupuser replication -c "IDENTIFY_SYSTEM;"
```

```
# postgresql.conf on standby – enable pg_basebackup from standby
hot_standby = on
max_wal_senders = 5
max_replication_slots = 5
```

Backup Formats and Options

Plain Format (Default)

```
# Plain: outputs directly to a directory (default)
pg_basebackup \
  --pgdata=/backup/base/20240115 \
  --format=plain \
  --wal-method=stream \
  --checkpoint=fast

# With gzip compression (-Z)
pg_basebackup \
  --pgdata=/backup/base/20240115 \
  --format=plain \
  --compress=gzip:5 \      # PostgreSQL 15+ syntax
  --wal-method=stream \
  --checkpoint=fast
```

Tar Format

```
# Tar format: outputs as tar files (one per tablespace)
pg_basebackup \
  --format=tar \
  --pgdata=/backup/base/ \
  --wal-method=stream \
  --checkpoint=fast

# Output files:
# /backup/base/base.tar          (main data directory)
# /backup/base/pg_wal.tar       (WAL files)
# /backup/base/<tablespace>.tar (if tablespaces exist)

# With compression
pg_basebackup \
  --format=tar \
  --compress=gzip:9 \
  --pgdata=/backup/base/ \
  --wal-method=stream

# Output: base.tar.gz, pg_wal.tar.gz

# Restore tar backup:
mkdir /restore/pgdata
cd /restore/pgdata
tar xf /backup/base/base.tar.gz
tar xf /backup/base/pg_wal.tar.gz -C pg_wal/
```

Additional Options

```
# Create replication slot during backup (for standby setup)
pg_basebackup \
  --pgdata=/backup/base/20240115 \
  --create-slot \
  --slot=backup_slot \
  --wal-method=stream \
  --checkpoint=fast

# Write recovery configuration (for standby setup)
pg_basebackup \
  --pgdata=/var/lib/postgresql/16/standby \
  --write-recovery-conf \           # Creates standby.signal + postgresql.auto.conf
  --wal-method=stream \
  --checkpoint=fast \
  --host=primary.example.com \
  --username=backupuser

# Limit bandwidth (protect network from backup load)
pg_basebackup \
  --pgdata=/backup/base/ \
  --max-rate=100M \               # Limit to 100MB/s
  --wal-method=stream \
  --checkpoint=fast
```

Monitoring Backup Progress

```
# Use --progress flag for real-time progress
pg_basebackup --progress --pgdata=/backup/base/ ...
# Output: 0/131072 kB (0%), 0/1 tablespace

# Monitor from PostgreSQL side
psql -c "SELECT pid, application_name, state, backup_start, backup_end
        FROM pg_stat_replication
        WHERE application_name = 'pg_basebackup';"

# Monitor I/O activity
iostat -x 2 5

# Check backup size in real-time
watch -n 5 'du -sh /backup/base/*'
```

Using Backup for Replication Setup

This is the most common use of `pg_basebackup` — setting up a new standby.

```

# On the standby server:
# 1. Take base backup from primary
sudo -u postgres pg_basebackup \
  --host=primary.example.com \
  --port=5432 \
  --username=replicator \
  --pgdata=/var/lib/postgresql/16/main \
  --wal-method=stream \
  --checkpoint=fast \
  --write-recovery-conf \           # Creates standby.signal and auto.conf
  --slot=standby1_slot \          # Create/use replication slot
  --create-slot \
  --progress \
  --verbose

# 2. Verify files created
ls -la /var/lib/postgresql/16/main/standby.signal
cat /var/lib/postgresql/16/main/postgresql.auto.conf
# Should contain:
# primary_conninfo = 'user=replicator host=primary.example.com port=5432 ...'
# primary_slot_name = 'standby1_slot'

# 3. Start standby
sudo systemctl start postgresql@16-main

# 4. Verify it's a standby
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
# Should return: t

```

Production Backup Script

```

#!/bin/bash
# pg_basebackup_production.sh

set -euo pipefail

# Configuration
PG_PRIMARY="${PG_PRIMARY:-primary.example.com}"
PG_STANDBY="${PG_STANDBY:-standby.example.com}" # Backup from standby if available
PG_PORT=5432
PG_REPL_USER=backupuser
BACKUP_BASE="/var/backups/pg_basebackup"
S3_BUCKET="s3://my-pg-backups/basebackup"
RETENTION_DAYS=7
DATE_STAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="${BACKUP_BASE}/${DATE_STAMP}"
LOG="/var/log/postgresql/basebackup_${DATE_STAMP}.log"

log() { echo "$(date '+%Y-%m-%d %H:%M:%S') $1" | tee -a "$LOG"; }

```

```

fail() { log "FATAL: $1"; exit 1; }

# Determine backup source (prefer standby)
if pg_isready -h "$PG_STANDBY" -p $PG_PORT -t 10; then
    BACKUP_SOURCE="$PG_STANDBY"
    log "Backing up from standby: $BACKUP_SOURCE"
else
    BACKUP_SOURCE="$PG_PRIMARY"
    log "WARNING: Standby unreachable, backing up from primary: $BACKUP_SOURCE"
fi

log "Starting base backup to $BACKUP_DIR"
mkdir -p "$BACKUP_DIR"

# Run pg_basebackup
START=$(date +%s)
pg_basebackup \
    --host="$BACKUP_SOURCE" \
    --port=$PG_PORT \
    --username="$PG_REPL_USER" \
    --pgdata="$BACKUP_DIR" \
    --format=tar \
    --compress=gzip:6 \
    --wal-method=stream \
    --checkpoint=fast \
    --label="prod-backup-${DATE_STAMP}" \
    --progress \
    --verbose \
    >> "$LOG" 2>&1 || fail "pg_basebackup failed"

END=$(date +%s)
DURATION=$((END - START))
BACKUP_SIZE=$(du -sh "$BACKUP_DIR" | cut -f1)
log "Backup complete in ${DURATION}s, size: ${BACKUP_SIZE}"

# Verify backup
[ -f "${BACKUP_DIR}/base.tar.gz" ] || fail "base.tar.gz not found"
gzip -t "${BACKUP_DIR}/base.tar.gz" || fail "base.tar.gz corrupted"
log "Backup integrity check passed"

# Upload to S3
log "Uploading to S3..."
aws s3 sync "$BACKUP_DIR" "${S3_BUCKET}/${DATE_STAMP}/" \
    --storage-class STANDARD_IA >> "$LOG" 2>&1 || log "WARNING: S3 upload failed"

# Cleanup old local backups
log "Cleaning up backups older than ${RETENTION_DAYS} days..."
find "$BACKUP_BASE" -maxdepth 1 -type d -mtime +${RETENTION_DAYS} \
    -exec rm -rf {} \; 2>/dev/null || true

log "Backup job completed successfully"

```

Common Mistakes

1. **Not using `--wal-method=stream`** — backup may be unusable if WAL is recycled
 2. **`max_wal_senders` too low** — backup fails with "too many WAL senders"
 3. **No replication privilege for backup user** — `pg_basebackup: error: could not connect`
 4. **Forgetting `--checkpoint=fast`** — long wait for natural checkpoint
 5. **No bandwidth limit** — backup saturates network during business hours
 6. **Backing up from primary with huge databases** — use standby instead
 7. **Not verifying backup files** — corrupted tar not detected
 8. **Insufficient disk space on backup target** — backup fails midway
 9. **Not monitoring backup duration trends** — longer backups indicate growth or issues
 10. **Using `pg_basebackup` as sole backup for large DBs** — use `pgBackRest` for incremental
-

Best Practices

1. Use `--wal-method=stream` (always)
 2. Run backups from **standby** to avoid primary impact
 3. Use `--checkpoint=fast` to avoid long waits
 4. Set `--max-rate` to limit backup bandwidth
 5. **Verify backup** after completion (`gzip -t, tar -tzvf`)
 6. **Upload to remote storage** (S3, GCS) after local backup
 7. **Monitor backup duration** — alert on abnormal times
 8. Use **replication slots** (`--create-slot`) when backup is for replication setup
 9. For **production PITR**, use `pgBackRest` instead (supports incremental)
 10. **Test restores monthly** — validate the backup is usable
-

Performance Considerations

```
# Benchmark backup speed
time pg_basebackup -h localhost -U backupuser -D /backup/test -Ft -Xs -P

# Typical rates:
# Local disk to local disk: 500MB/s - 2GB/s (I/O bound)
# Over 1GbE network: ~100MB/s
# Over 10GbE: ~1GB/s
# S3-based (via pgBackRest): 100-500MB/s

# Tune for faster backups:
pg_basebackup \
  --max-rate=500M \      # Allow up to 500MB/s
  --checkpoint=fast \    # Immediate checkpoint
  -Z 1 \                # Lower compression (faster, larger files)
  ...

# Parallel compression (PostgreSQL 15+)
pg_basebackup \
  --compress=gzip:5,workers=4 \ # 4 compression workers
  ...
```

Interview Questions

Q1: What does `pg_basebackup` do and how is it different from `pg_dump`?

A: `pg_basebackup` takes a physical binary copy of the entire PostgreSQL data directory using the replication protocol. It produces an exact copy of the database files (data blocks) at a consistent point in time. `pg_dump` exports data as SQL (or custom format), is selective, and is cross-version compatible. `pg_basebackup`: faster restore, enables PITR, must same PG version. `pg_dump`: slower, selective, cross-version.

Q2: What is `--wal-method=stream` and why is it critical?

A: Without streaming WAL, `pg_basebackup` takes a backup but the WAL generated during the backup may be recycled by the primary before the backup is complete. The backup would then be unrestoreable (missing WAL needed to make it consistent). `--wal-method=stream` opens a second connection to continuously stream WAL during the backup, ensuring all necessary WAL is captured. It requires `max_wal_senders >= 2`.

Q3: Why would you run `pg_basebackup` against a standby instead of the primary?

A: To reduce load on the primary. The backup involves reading the entire data directory, which is I/O intensive. On a hot primary handling production traffic, this can increase I/O wait and slow down queries. A standby serves the same data and can handle the backup I/O without affecting production. Requires `hot_standby = on` and `max_wal_senders` on the standby.

Q4: What files does `--write-recovery-conf` create?

A: (PostgreSQL 12+) It creates two files: (1) `standby.signal` — an empty file that signals PostgreSQL to start in standby mode. (2) `postgresql.auto.conf` — with `primary_conninfo` pointing to the source server and optionally `primary_slot_name`. In PostgreSQL 11 and earlier, it created `recovery.conf`.

Q5: What is the difference between `--format=plain` and `--format=tar`?

A: Plain (default): copies files as-is into the target directory, preserving the directory structure. You get a ready-to-use data directory. Tar: packages files into tar archives (one per tablespace). Easier to upload to remote storage or compress. You need to untar before use. Plain is easier for immediate standby setup; tar is better for archival/cloud storage.

Q6: How does `pg_basebackup` ensure backup consistency?

A: It starts by sending a CHECKPOINT command to ensure a full checkpoint is written. The backup is based on this checkpoint LSN. The backup includes all WAL generated after the checkpoint (either via stream or fetch) to make the backup consistent. The `backup_label` file in the backup records the checkpoint LSN; at restore time, PostgreSQL replays WAL from that point to reach a consistent state.

Exercises and Solutions

Exercise 1: Complete Standby Setup via `pg_basebackup`

```
#!/bin/bash
# setup_standby_from_basebackup.sh

PRIMARY_HOST=primary.example.com
STANDBY_DATA=/var/lib/postgresql/16/main
REPL_USER=replicator
```

```
SLOT_NAME=standby1

# Ensure clean state
sudo systemctl stop postgresql@16-main 2>/dev/null || true
sudo -u postgres rm -rf "$STANDBY_DATA"
mkdir -p "$STANDBY_DATA"

# Take base backup with recovery conf
sudo -u postgres pg_basebackup \
  --host="$PRIMARY_HOST" \
  --username="$REPL_USER" \
  --pgdata="$STANDBY_DATA" \
  --wal-method=stream \
  --checkpoint=fast \
  --create-slot \
  --slot="$SLOT_NAME" \
  --write-recovery-conf \
  --progress \
  --verbose

# Verify
ls "$STANDBY_DATA/standby.signal" # Must exist
grep "primary_conninfo" "$STANDBY_DATA/postgresql.auto.conf"

# Start standby
sudo systemctl start postgresql@16-main
sleep 5
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
```

Cross-References

- [01 backup strategies.md](#) — When to use pg_basebackup
- [04_pitr.md](#) — PITR using pg_basebackup + WAL
- [05 wal archiving.md](#) — WAL archiving for PITR
- [06 pgbackrest.md](#) — Production alternative with incremental support
- [../13 Replication HA/02 streaming_replication.md](#) — Using pg_basebackup for replication setup